

Tiny Machine Learning

Práctica 3

Control de una casa inteligente con un SLM en Raspberry Pi

Romero de la Cruz Alejandro

Ruvalcaba Tamés Led Eduardo

Tomás Díaz Víctor Manuel

Gudiño Tapia Daniel Raul

3 de diciembre de 2025

Asignatura: Tiny Machine Learning

Dispositivo: Raspberry Pi 400 (Debian 13, aarch64)

Índice

1. Introducción	3
2. Objetivos de la práctica	4
3. Marco teórico	5
3.1. Small Language Models (SLM) en entornos TinyML	5
3.2. Ejecución de SLM en Raspberry Pi	5
3.3. GPIO y sensores en la Raspberry Pi	6
4. Desarrollo	7
4.1. Preparación de la Raspberry Pi 400	7
4.2. Instalación de dependencias y <code>llama.cpp</code>	7
4.3. Ejecución del SLM en la Raspberry	8
4.4. Control de GPIO para la “casa inteligente”	8
4.5. Diseño de la API semántica con el SLM	9
4.5.1. Comandos simbólicos	9
4.5.2. Ejecución de comandos	10
5. Resultados	11
6. Conclusiones	14

1. Introducción

Los modelos de lenguaje de gran tamaño (LLM, *Large Language Models*) han demostrado ser extremadamente poderosos para tareas de generación de texto, razonamiento, asistencia y agentes conversacionales. Sin embargo, su tamaño (decenas o cientos de miles de millones de parámetros) los vuelve muy costosos en términos de memoria, energía y cómputo, lo cual dificulta su uso en dispositivos embebidos o de bajo consumo.

En este contexto surgen los **Small Language Models (SLM)**, es decir, modelos de lenguaje mucho más pequeños (del orden de 1–3 mil millones de parámetros e incluso menos), pensados para ejecutarse en dispositivos modestos como:

- Raspberry Pi (3/4/400/5).
- Micro PCs y mini servidores.
- Aceleradores ligeros (NPUs, TPUs pequeñas, etc.).

Ejemplos de SLMs actuales incluyen variantes pequeñas de familias como LLaMA, Gemma, Phi, Qwen, entre otras, muchas de ellas cuantizadas (por ejemplo, en 4 bits) para reducir aún más el uso de memoria. Aunque sacrifican algo de calidad respecto a los LLMs más grandes, ofrecen ventajas importantes:

- **Baja latencia local:** el modelo corre en el propio dispositivo, sin depender de la nube.
- **Privacidad:** los datos no salen del dispositivo.
- **Costos reducidos:** no se paga cómputo en servidores externos.
- **Integración con hardware:** el modelo puede controlar sensores y actuadores directamente (GPIO, I2C, SPI, etc.).

En esta práctica de **Tiny Machine Learning** se explora justamente este escenario: ejecutar un SLM dentro de una **Raspberry Pi 400** y usarlo como “cerebro” de un sistema domótico sencillo, donde el usuario pueda escribir frases coloquiales como:

“Prende la luz del cuarto”

“Apaga la cocina”

“Dime la temperatura”

y el sistema traduzca esas frases en acciones sobre los GPIO y la lectura de sensores.

Originalmente, el enunciado de la práctica planteaba un escenario de **Biblioteca** (secciones de libros, sistemas de iluminación por área y un sistema de deshumificación para proteger colecciones especiales). En nuestro caso, re-interpretamos la actividad como una **casa inteligente**, para simular el comportamiento de asistentes comerciales tipo Alexa o Google Home, pero manteniendo la idea central: un SLM que interpreta lenguaje natural y lo traduce en órdenes sobre un sistema de control y sensado.

La arquitectura general es la misma:

- Zonas (o secciones) controladas por GPIO.
- Sensores (temperatura/humedad) leídos desde la Raspberry.
- Un SLM que recibe texto coloquial y lo convierte en comandos simples (encender, apagar, consultar estado).

2. Objetivos de la práctica

El enunciado original pedía, de forma resumida, lo siguiente:

1. Ejecutar y evaluar varios SLM en la Raspberry Pi y escoger el más adecuado.
2. Controlar sistemas de iluminación (por ejemplo, en una biblioteca) con frases coloquiales: “apaga la luz en la sección de Historia”.
3. Integrar un sensor de temperatura (por ejemplo, DHT11) y un sistema de deshumidificación controlado por lenguaje natural: “está lloviendo, ¿corren riesgo los libros de Cálculo?”.
4. Diseñar una actividad extra que aproveche el SLM.

En nuestro caso adaptamos el escenario a una **casa inteligente**:

- Zonas: **cuarto, cocina y baño**, cada una mapeada a un GPIO.
- Sensor de temperatura: se utiliza la **temperatura de la CPU** de la Raspberry Pi como sustituto, con la idea de que el código se puede modificar fácilmente para leer un DHT11 físico.
- Interfaz coloquial: el usuario escribe frases tipo “prende la luz del cuarto” y el sistema interpreta y ejecuta la orden.

Motivación: la meta es simular un asistente tipo Alexa completamente local, donde el SLM corre en la propia Raspberry y puede tomar decisiones simples sobre la casa (o biblioteca) usando tanto el contexto conversacional como la información de sensores físicos.

Los objetivos específicos de nuestra implementación fueron:

- a) Configurar la Raspberry Pi 400, su sistema operativo (Debian 13, 64 bits) y el entorno de desarrollo.
- b) Compilar y ejecutar un SLM (usando `llama.cpp`) directamente en la Raspberry.
- c) Diseñar una API sencilla (en Python) que:
 - reciba texto coloquial del usuario,
 - lo traduzca a comandos simbólicos usando el SLM,
 - y aplique esos comandos sobre los GPIO y sobre la lectura de temperatura.
- d) Demostrar el funcionamiento con un escenario de casa inteligente y capturas/fotografías del sistema.

3. Marco teórico

3.1. Small Language Models (SLM) en entornos TinyML

Un **modelo de lenguaje** moderno se entrena para predecir la siguiente palabra (o token) en una secuencia. Los modelos grandes (LLM) tienen cientos de capas y decenas de miles de millones de parámetros, lo que los hace muy costosos de ejecutar.

Los **Small Language Models (SLM)** son variantes reducidas, típicamente entre ~500M y 3B parámetros, diseñadas para:

- tener menor uso de RAM,
- consumir menos energía,
- mantener una calidad razonable en tareas cotidianas de chat, clasificación, extracción de información, etc.

Algunos ejemplos representativos son:

- Versiones pequeñas de **LLaMA 3** (por ejemplo, 1B parámetros).
- Modelos como **Gemma** en versiones 1B–2B.
- Modelos ligeros tipo **Phi** y **Qwen** en tamaños pequeños.

Estos modelos se suelen cuantizar (por ejemplo, a 4 bits) y exportar a formatos optimizados (como GGUF) para ser ejecutados por librerías como `llama.cpp` en CPU.

3.2. Ejecución de SLM en Raspberry Pi

La **Raspberry Pi 400** es, internamente, muy similar a una Raspberry Pi 4 con una CPU ARM de 64 bits (Cortex-A72). Aunque no tiene una GPU potente para inferencia, sí permite:

- Ejecutar un SLM pequeño (1B parámetros cuantizado) en CPU.
- Mantener latencias aceptables para conversaciones cortas.
- Integrar el modelo con el ecosistema Linux (Python, GPIO, servicios, etc.).

En esta práctica se utilizó:

- Sistema operativo: **Debian GNU/Linux 13 (trixie), aarch64**.
- Biblioteca de inferencia: `llama.cpp`.
- Modelo SLM: una variante pequeña estilo **Gemma 3 1B Instruct** en formato GGUF, descargada y ejecutada directamente en la Raspberry.

3.3. GPIO y sensores en la Raspberry Pi

La Raspberry Pi expone pines GPIO (entrada/salida digital) con los que se puede:

- Encender y apagar LEDs, relevadores y sistemas de iluminación.
- Leer botones, sensores simples de nivel lógico, etc.

En Python, bibliotecas como `gpiozero` simplifican el control de estos pines:

- `LED(17)` representa un LED conectado al GPIO17 (modo BCM).
- Métodos como `on()` / `off()` permiten controlar el estado de cada salida.

Para sensores de temperatura como el **DHT11**, es común usar una biblioteca específica (por ejemplo, `Adafruit_DHT`) para leer la temperatura y humedad desde un GPIO. En esta práctica no se conectó físicamente un DHT11, pero se diseñó el sistema de forma que:

- Se utilizó la **temperatura de la CPU** de la Raspberry (archivo `/sys/class/thermal/thermal_zone0/temp`) como *sensor proxy*.
- El código queda preparado para reemplazar la función de lectura de temperatura por una versión basada en DHT11 sin modificar la lógica del SLM ni del control de GPIO.

4. Desarrollo

4.1. Preparación de la Raspberry Pi 400

Primero se verificó el modelo y sistema operativo de la Raspberry:

- `cat /sys/firmware/devicetree/base/model`
→ Raspberry Pi 400 Rev 1.1
- `uname -a`
→ kernel Linux 6.12, arquitectura aarch64.
- `cat /etc/os-release`
→ Debian GNU/Linux 13 (trixie).

Se trabajó directamente en la terminal de la Raspberry (o vía SSH), pero siempre **dentro** del propio dispositivo. No se utilizó ningún servidor externo para la inferencia del modelo; toda la ejecución del SLM fue local.

4.2. Instalación de dependencias y llama.cpp

Se instalaron las herramientas básicas de compilación:

```
sudo apt update
sudo apt install -y git build-essential cmake \
                    libcurl4-openssl-dev python3-gpiozero \
                    python3-requests
```

Luego se clonó el repositorio de llama.cpp y se compiló:

```
cd ~
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp

cmake -B build
cmake --build build --config Release
```

Al final de este proceso aparecieron binarios como:

- `build/bin/llama-cli`
- `build/bin/llama-server`

que permiten usar el modelo desde la terminal o como servicio HTTP.

4.3. Ejecución del SLM en la Raspberry

Para probar rápidamente el modelo desde consola se utilizó el modo `llama-cli` con descarga directa desde Hugging Face (sólo la primera vez):

```
cd ~/llama.cpp
./build/bin/llama-cli \
    -hf ggml-org/gemma-3-1b-it-GGUF \
    -p "Respóndeme en español: ¿qué puedes hacer en una Raspberry Pi?" \
    -n 64
```

Una vez comprobado que el modelo funcionaba, se decidió usar `llama-server` para evitar tener que cargar el modelo en cada llamada. El servidor se lanzó así:

```
cd ~/llama.cpp
./build/bin/llama-server \
    -hf ggml-org/gemma-3-1b-it-GGUF \
    --host 127.0.0.1 \
    --port 8080 \
    -c 256 \
    --threads 3 &
```

Con esto, el SLM queda escuchando peticiones HTTP en `http://127.0.0.1:8080`, usando una API compatible con `/v1/chat/completions`. El **modelo vive en la RAM de la Raspberry** mientras el servidor está activo.

4.4. Control de GPIO para la “casa inteligente”

Para simular la casa inteligente se eligieron tres GPIO:

- GPIO17 (BCM) → Luz del **cuarto**.
- GPIO27 (BCM) → Luz de la **cocina**.
- GPIO22 (BCM) → Luz del **baño**.

Cada salida se conectó a un LED (o relevador) con su resistencia y referencia a GND. En Python, se utilizó `gpiozero` para controlar estas salidas:

```
from gpiozero import LED

led_cuarto = LED(17)
led_cocina = LED(27)
led_bano   = LED(22)
```

La lectura de temperatura se resolvió mediante la temperatura interna de la CPU:

```
def leer_temperatura_cpu():
    with open("/sys/class/thermal/thermal_zone0/temp", "r") as f:
        millic = int(f.read().strip())
    return millic / 1000.0 # grados Celsius
```

En el contexto de la práctica, esta función representa al sensor físico (como un DHT11). En un escenario real, bastaría con sustituirla por una función que use la biblioteca del DHT11 para leer temperatura y humedad desde un GPIO, manteniendo toda la lógica del SLM y de la “casa inteligente” sin cambios.

4.5. Diseño de la API semántica con el SLM

La API en Python se organizó en dos capas:

1. **Capa semántica (SLM):** recibe el texto del usuario y lo convierte en un *comando simbólico*.
2. **Capa de control (GPIO/sensores):** recibe el comando simbólico y actúa sobre los pines y la lectura de temperatura.

4.5.1. Comandos simbólicos

Se definieron comandos simbólicos como:

- CUARTO_ON, CUARTO_OFF
- COCINA_ON, COCINA_OFF
- BANO_ON, BANO_OFF
- STATUS (consultar estado de todas las luces)
- TEMP (consultar temperatura)

La función `pedir_comando_llama()` construye un mensaje de sistema que explica al SLM cómo debe mapear el lenguaje coloquial a uno de esos comandos, por ejemplo:

“Usuario: prende la luz del baño → Respuesta: BANO_ON”

y luego hace una petición HTTP al servidor:

```
payload = {
    "model": MODEL_NAME,
    "messages": [
        {"role": "system", "content": system_msg},
        {"role": "user", "content": texto_usuario},
    ],
    "max_tokens": 8,
    "temperature": 0.2,
}

resp = requests.post(LLAMA_URL, json=payload, timeout=60)
data = resp.json()
comando = data["choices"][0]["message"]["content"].strip().split()[0].upper()
```

Si el SLM responde algo inesperado, se fuerza un comando por defecto (STATUS) para evitar errores.

4.5.2. Ejecución de comandos

La función `ejecutar_comando(comando)` se encarga de traducir el comando simbólico a acciones sobre los GPIO y la temperatura:

- Encender o apagar LEDs asociados a cuarto, cocina y baño.
- Imprimir en consola el estado actual de cada zona.
- Leer y mostrar la temperatura actual de la CPU.

De esta forma, la **lógica del hardware** está desacoplada de la **interpretación de lenguaje natural**. Si se quisiera regresar al escenario de la biblioteca, bastaría con cambiar los nombres de las zonas (por ejemplo, *Historia*, *Cálculo*, *Literatura*) y los mensajes de estado, manteniendo idéntica la estructura del software.

5. Resultados

Se logró:

- Compilar y ejecutar un SLM pequeño (Gemma 3 1B-like) directamente en la Raspberry Pi 400 usando `llama.cpp`.
- Exponer el modelo vía `llama-server` con una API HTTP tipo `/v1/chat/completions`.
- Implementar un script en Python que:
 - envía frases coloquiales a la API del SLM,
 - recibe un comando simbólico,
 - y controla tres GPIO (cuarto, cocina, baño) simulando un sistema domótico.
- Integrar una lectura de temperatura (CPU de la Raspberry) en el mismo flujo, pensada como reemplazo directo de un sensor físico DHT11.

En la práctica, se pudo escribir órdenes como:

- “*Prende la luz del cuarto*” → CUARTO_ON
- “*Apaga la cocina*” → COCINA_OFF
- “*¿Cómo están las luces?*” → STATUS
- “*Dime la temperatura*” → TEMP

A continuación se muestran algunas capturas de pantalla representativas del sistema en ejecución.

```
=== ESTADO DE LA CASA ===
Cuarto : ENCENDIDA
Cocina : ENCENDIDA
Baño   : APAGADA
=====
srv_log_server_r: request: POST /v1/chat/completions 127.0.0.1 200

Escribe una orden para la casa (Ctrl+C para salir): dime la temperatura
Temperatura actual (simulada): 26.6 °C

Escribe una orden para la casa (Ctrl+C para salir): dame la temperatura
Temperatura actual (simulada): 39.8 °C

Escribe una orden para la casa (Ctrl+C para salir): prende el baño
Luz del baño ENCENDIDA.

Escribe una orden para la casa (Ctrl+C para salir): estado de la casa
srv_params_from: Chat format: Content-only
slot_get_availabl: id 3 | task -1 | selected slot by LCP similarity, sim_best = 0.962 (> 0.100 thold), f_keep = 0.950
slot_launch_slot: id 3 | task -1 | sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
slot_launch_slot: id 3 | task 12 | processing task
slot_update_slots: id 3 | task 12 | new prompt, n_ctx_slot = 4896, n_keep = 0, task.n_tokens = 235
slot_update_slots: id 3 | task 12 | n_tokens = 226, memory_seq_rm [226, end]
slot_update_slots: id 3 | task 12 | prompt processing progress, n_tokens = 235, batch.n_tokens = 9, progress = 1.000000
slot_update_slots: id 3 | task 12 | prompt done, n_tokens = 235, batch.n_tokens = 9
slot_print_timing: id 3 | task 12 |
prompt eval time = 840.21 ms / 9 tokens ( 93.36 ms per token, 10.71 tokens per second)
eval time = 474.99 ms / 3 tokens ( 158.33 ms per token, 6.32 tokens per second)
total time = 1315.19 ms / 12 tokens
slot_release: id 3 | task 12 | stop processing: n_tokens = 237, truncated = 0
srv_update_slots: all slots are idle
srv_log_server_r: request: POST /v1/chat/completions 127.0.0.1 200

=== ESTADO DE LA CASA ===
Cuarto : ENCENDIDA
Cocina : ENCENDIDA
Baño   : ENCENDIDA
=====
```

Figura 1: Temperatura simulada, y cuartos encendidos.

```

=== ESTADO DE LA CASA ===
Cuarto : ENCENDIDA
Cocina : ENCENDIDA
Baño   : APAGADA
=====
srv log_server_r: request: POST /v1/chat/completions 127.0.0.1 200

Escribe una orden para la casa (Ctrl+C para salir):

```

Figura 2: Estado de la casa mostrando cuarto y cocina encendidos, baño apagado; se observa también el log del servidor al recibir la petición.

```

Escribe una orden para la casa (Ctrl+C para salir): dime la temperatura
La temperatura es: 44.3 °C

Escribe una orden para la casa (Ctrl+C para salir): give me the temperature
srv params_from.: Chat format: Content-only
slot get_availabl.: id 3 | task -1 | selected slot by LCP similarity, sim_best = 0.963 (> 0.100 thold), f_keep = 0.947
slot launch_slot.: id 3 | task -1 | sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
slot launch_slot.: id 3 | task 8 | processing task
slot update_slots.: id 3 | task 8 | new prompt, n_ctx_slot = 4096, n_keep = 0, task.n_tokens = 241
slot update_slots.: id 3 | task 8 | n_tokens = 232, memory_seq_rm [232, end]
slot update_slots.: id 3 | task 8 | prompt processing progress, n_tokens = 241, batch.n_tokens = 9, progress = 1.000000
slot update_slots.: id 3 | task 8 | prompt done, n_tokens = 241, batch.n_tokens = 9
slot print_timing.: id 3 | task 8 |
prompt eval time = 846.46 ms / 9 tokens ( 94.05 ms per token, 10.63 tokens per second)
eval time = 475.45 ms / 3 tokens ( 158.48 ms per token, 6.31 tokens per second)
total time = 1321.90 ms / 12 tokens
slot release.: id 3 | task 8 | stop processing: n_tokens = 243, truncated = 0
srv update_slots: all slots are idle
srv log_server_r: request: POST /v1/chat/completions 127.0.0.1 200
La temperatura es: 42.8 °C

```

Figura 3: Petición en inglés (*give me the temperature*) procesada por el SLM y respuesta con la temperatura leída en la Raspberry.

```

Escribe una orden para la casa (Ctrl+C para salir): apaga el cuarto
srv params_from.: Chat format: Content-only
slot get_availabl.: id 3 | task -1 | selected slot by LCP similarity, sim_best = 0.970 (> 0.100 thold), f_keep = 0.950
slot launch_slot.: id 3 | task -1 | sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
slot launch_slot.: id 3 | task 35 | processing task
slot update_slots.: id 3 | task 35 | new prompt, n_ctx_slot = 4096, n_keep = 0, task.n_tokens = 235
slot update_slots.: id 3 | task 35 | n_tokens = 228, memory_seq_rm [228, end]
slot update_slots.: id 3 | task 35 | prompt processing progress, n_tokens = 235, batch.n_tokens = 7, progress = 1.000000
slot update_slots.: id 3 | task 35 | prompt done, n_tokens = 235, batch.n_tokens = 7
slot print_timing.: id 3 | task 35 |
prompt eval time = 695.19 ms / 7 tokens ( 99.31 ms per token, 10.07 tokens per second)
eval time = 1420.64 ms / 7 tokens ( 202.95 ms per token, 4.91 tokens per second)
total time = 2119.83 ms / 14 tokens
slot release.: id 3 | task 35 | stop processing: n_tokens = 241, truncated = 0
srv update_slots: all slots are idle
srv log_server_r: request: POST /v1/chat/completions 127.0.0.1 200
Luz del cuarto APAGADA.

Escribe una orden para la casa (Ctrl+C para salir): apaga el baño
srv params_from.: Chat format: Content-only
slot get_availabl.: id 3 | task -1 | selected slot by LCP similarity, sim_best = 0.974 (> 0.100 thold), f_keep = 0.950
slot launch_slot.: id 3 | task -1 | sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
slot launch_slot.: id 3 | task 43 | processing task
slot update_slots.: id 3 | task 43 | new prompt, n_ctx_slot = 4096, n_keep = 0, task.n_tokens = 235
slot update_slots.: id 3 | task 43 | n_tokens = 229, memory_seq_rm [229, end]
slot update_slots.: id 3 | task 43 | prompt processing progress, n_tokens = 235, batch.n_tokens = 6, progress = 1.000000
slot update_slots.: id 3 | task 43 | prompt done, n_tokens = 235, batch.n_tokens = 6
slot print_timing.: id 3 | task 43 |
prompt eval time = 623.18 ms / 6 tokens ( 103.86 ms per token, 9.63 tokens per second)
eval time = 1185.75 ms / 6 tokens ( 197.62 ms per token, 5.06 tokens per second)
total time = 1808.93 ms / 12 tokens
slot release.: id 3 | task 43 | stop processing: n_tokens = 240, truncated = 0
srv update_slots: all slots are idle
srv log_server_r: request: POST /v1/chat/completions 127.0.0.1 200
Luz del baño APAGADA.

Escribe una orden para la casa (Ctrl+C para salir): estado de la casa

=== ESTADO DE LA CASA ===
Cuarto : APAGADA
Cocina : APAGADA
Baño   : APAGADA
=====

```

Figura 4: Órdenes para apagar el cuarto y el baño; al final se muestra el estado de la casa con todas las luces apagadas.

```

llamas@raspberrypi:~/llama.cpp $ cd ~/proyectos_gpio
python3 casa_inteligente.py
Casa inteligente con Llama
Ejemplos de órdenes:
- prende la luz del cuarto
- apaga la cocina
- enciende el baño
- cómo están las luces?
- dime la temperatura

Escribe una orden para la casa (Ctrl+C para salir): prende la luz del cuarto
Luz del cuarto ENCENDIDA.

Escribe una orden para la casa (Ctrl+C para salir): prende la luz de la cocina
srv_params_from_: Chat format: Content-only
slot_get_availabl: id 3 | task -1 | selected slot by LRU, t_last = -1
slot_launch_slot_: id 3 | task -1 | sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
slot_launch_slot_: id 3 | task 0 | processing task
slot_update_slots: id 3 | task 0 | new prompt, n_ctx_slot = 4096, n_keep = 0, task.n_tokens = 238
slot_update_slots: id 3 | task 0 | n_tokens = 0, memory_seq_rm [0, end)
slot_update_slots: id 3 | task 0 | prompt processing progress, n_tokens = 174, batch.n_tokens = 174, progress = 0.731892
slot_update_slots: id 3 | task 0 | n_tokens = 174, memory_seq_rm [174, end)
slot_update_slots: id 3 | task 0 | prompt processing progress, n_tokens = 238, batch.n_tokens = 64, progress = 1.000000
slot_update_slots: id 3 | task 0 | prompt done, n_tokens = 238, batch.n_tokens = 64
slot_update_slots: id 3 | task 0 | created context checkpoint 1 of 8 (pos_min = 0, pos_max = 173, size = 3.741 MiB)
slot_print_timing: id 3 | task 0 |
prompt eval time = 17371.06 ms / 238 tokens ( 72.99 ms per token, 13.70 tokens per second)
eval time = 1179.62 ms / 6 tokens ( 196.60 ms per token, 5.09 tokens per second)
total time = 18550.68 ms / 244 tokens
slot_release: id 3 | task 0 | stop processing: n_tokens = 243, truncated = 0
srv_update_slots: all slots are idle
srv_log_server_r: request: POST /v1/chat/completions 127.0.0.1 280
Luz de la cocina ENCENDIDA.

Escribe una orden para la casa (Ctrl+C para salir): como van las luces?
srv_params_from_: Chat format: Content-only
slot_get_availabl: id 3 | task -1 | selected slot by LCP similarity, sim_best = 0.958 (> 0.100 thold), f_keep = 0.930
slot_launch_slot_: id 3 | task -1 | sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
slot_launch_slot_: id 3 | task 8 | processing task
slot_update_slots: id 3 | task 8 | new prompt, n_ctx_slot = 4096, n_keep = 0, task.n_tokens = 236
slot_update_slots: id 3 | task 8 | n_tokens = 226, memory_seq_rm [226, end)
slot_update_slots: id 3 | task 8 | prompt processing progress, n_tokens = 236, batch.n_tokens = 10, progress = 1.000000
slot_update_slots: id 3 | task 8 | prompt done, n_tokens = 236, batch.n_tokens = 10
slot_print_timing: id 3 | task 8 |
prompt eval time = 898.00 ms / 10 tokens ( 89.80 ms per token, 11.14 tokens per second)
eval time = 476.69 ms / 3 tokens ( 158.90 ms per token, 6.29 tokens per second)
total time = 1374.68 ms / 13 tokens
slot_release: id 3 | task 8 | stop processing: n_tokens = 238, truncated = 0
srv_update_slots: all slots are idle

=== ESTADO DE LA CASA ===

```

Figura 5: Encendido de la luz del cuarto y de la cocina, seguido de una consulta “cómo van las luces” y el estado consolidado.

6. Conclusiones

Esta práctica permitió comprobar que:

- Es posible ejecutar un **Small Language Model** en una Raspberry Pi 400 sin depender de la nube, siempre que se elija un modelo lo suficientemente pequeño y cuantizado.
- La combinación **SLM + GPIO** convierte a la Raspberry en una especie de *asistente inteligente local*, capaz de traducir lenguaje coloquial a acciones concretas sobre hardware.
- Separar la lógica en:
 1. **Interpretación** (SLM que produce un comando simbólico).
 2. **Control** (Python que ejecuta el comando sobre GPIO y sensores).

hace que el sistema sea fácilmente extensible a otros escenarios (biblioteca, laboratorio, invernadero, etc.).

- Usar la temperatura de la CPU como sensor fue suficiente para validar el flujo, y la función se puede sustituir por un lector real de DHT11 sin modificar el resto del código.

En cuanto al análisis de SLMs, aunque en la práctica sólo se implementó de manera profunda un modelo pequeño basado en Gemma, el ejercicio permitió discutir las características que debe tener un modelo para ser adecuado en un entorno TinyML:

- Tamaño reducido (1–3B parámetros).
- Cuantización ligera (4 bits).
- Soporte en librerías eficientes como `llama.cpp`.
- Buen compromiso entre calidad de respuesta y consumo de memoria/tiempo.

Finalmente, el cambio de escenario de **biblioteca** a **casa inteligente** se justificó como una forma más intuitiva de simular un asistente tipo Alexa, sin perder los elementos clave del enunciado: control de iluminación por zonas, dependencia de condiciones ambientales (temperatura) y uso de lenguaje coloquial.

Como trabajo futuro se podrían explorar:

- Uso de un DHT11 real y otros sensores (humedad, luminosidad, presencia).
- Integración de varios SLMs y comparación cuantitativa de latencia y calidad.
- Diálogos más complejos donde el SLM recuerde el contexto (por ejemplo, “apaga todo lo que prendiste hace rato”).
- Incorporación de un sistema de registro de eventos (log) para auditoría y depuración.

Anexo A. Código fuente de la casa inteligente

A continuación se muestra el código completo en Python utilizado para controlar la “casa inteligente” con el SLM ejecutándose en la Raspberry Pi 400.

```
from gpiozero import LED
import requests

# ===== CONFIGURACION DE GPIO =====
# Cuarto:  GPIO17 (BCM)  -> pin fisico 11
# Cocina:  GPIO27 (BCM)  -> pin fisico 13
# Bano:    GPIO22 (BCM)  -> pin fisico 15

led_cuarto = LED(17)
led_cocina = LED(27)
led_bano   = LED(22)

# ===== CONFIGURACION DE LLAMA-SERVER =====
LLAMA_URL = "http://127.0.0.1:8080/v1/chat/completions"
MODEL_NAME = "ggml-org/gemma-3-1b-it-GGUF"

COMANDOS_VALIDOS = {
    "CUARTO_ON",
    "CUARTO_OFF",
    "COCINA_ON",
    "COCINA_OFF",
    "BANO_ON",
    "BANO_OFF",
    "STATUS",
    "TEMP",
}

def leer_temperatura_cpu():
    """
    Lee la temperatura de la CPU de la Raspberry Pi en grados Celsius,
    usando /sys/class/thermal/thermal_zone0/temp.
    """
    try:
        with open("/sys/class/thermal/thermal_zone0/temp", "r") as f:
            millic = int(f.read().strip())
            return millic / 1000.0
    except Exception:
        return None

def pedir_comando_llama(texto_usuario: str) -> str:
    """
    Envía el texto del usuario al SLM y obtiene un comando simbolico.
    Comandos posibles:
        CUARTO_ON, CUARTO_OFF,
```

```

        COCINA_ON, COCINA_OFF,
        BANO_ON, BANO_OFF,
        STATUS, TEMP
    """
    system_msg = """
Eres el PARSEER de comandos de una casa inteligente.
Tienes tres luces: CUARTO, COCINA y BANO, y un sensor de temperatura.

Tu trabajo es leer ordenes en espanol del usuario y convertirlas
en EXACTAMENTE UNO de estos comandos (en mayusculas):

CUARTO_ON    -> encender luz del cuarto
CUARTO_OFF   -> apagar luz del cuarto
COCINA_ON    -> encender luz de la cocina
COCINA_OFF   -> apagar luz de la cocina
BANO_ON      -> encender luz del bano
BANO_OFF     -> apagar luz del bano
STATUS       -> el usuario quiere saber el estado de todas las luces
TEMP        -> el usuario quiere saber la temperatura

RESPONDE SIEMPRE con solo uno de esos comandos, sin explicacion extra.
""".strip()

    payload = {
        "model": MODEL_NAME,
        "messages": [
            {"role": "system", "content": system_msg},
            {"role": "user", "content": texto_usuario},
        ],
        "max_tokens": 8,
        "temperature": 0.2,
    }

    resp = requests.post(LLAMA_URL, json=payload, timeout=60)
    resp.raise_for_status()
    data = resp.json()

    content = data["choices"][0]["message"]["content"].strip()
    comando = content.split()[0].upper()

    if comando not in COMANDOS_VALIDOS:
        comando = "STATUS"

    return comando

def imprimir_estado():
    def txt(led):
        return "ENCENDIDA" if led.is_lit else "APAGADA"

```

```

print("\n=== ESTADO DE LA CASA ===")
print(f"Cuarto : {txt(led_cuarto)}")
print(f"Cocina : {txt(led_cocina)}")
print(f"Bano   : {txt(led_bano)}")
print("=====\n")

def ejecutar_comando(comando: str):
    """
    Ejecuta el comando simbolico sobre los GPIOs
    y muestra el resultado en la terminal.
    """
    if comando == "CUARTO_ON":
        led_cuarto.on()
        print("Luz del cuarto ENCENDIDA.\n")

    elif comando == "CUARTO_OFF":
        led_cuarto.off()
        print("Luz del cuarto APAGADA.\n")

    elif comando == "COCINA_ON":
        led_cocina.on()
        print("Luz de la cocina ENCENDIDA.\n")

    elif comando == "COCINA_OFF":
        led_cocina.off()
        print("Luz de la cocina APAGADA.\n")

    elif comando == "BANO_ON":
        led_bano.on()
        print("Luz del bano ENCENDIDA.\n")

    elif comando == "BANO_OFF":
        led_bano.off()
        print("Luz del bano APAGADA.\n")

    elif comando == "STATUS":
        imprimir_estado()

    elif comando == "TEMP":
        temp = leer_temperatura_cpu()
        if temp is None:
            print("No se pudo leer la temperatura.\n")
        else:
            print(f"Temperatura CPU Raspberry: {temp:.1f} C\n")

    else:
        print(f"(Comando desconocido: {comando})\n")

```

```

if __name__ == "__main__":
    print("Casa inteligente con Llama")
    print("Ejemplos de ordenes:")
    print(" - prende la luz del cuarto")
    print(" - apaga la cocina")
    print(" - enciende el bano")
    print(" - como estan las luces?")
    print(" - dime la temperatura\n")

    try:
        while True:
            try:
                texto = input(
                    "Escribe una orden para la casa (Ctrl+C para salir): "
                )
            except EOFError:
                break

            if not texto.strip():
                continue

            try:
                comando = pedir_comando_llama(texto)
            except Exception as e:
                print(f"Error hablando con Llama: {e}")
                continue

            ejecutar_comando(comando)

    except KeyboardInterrupt:
        print("\nSaliendo... apagando luces.")
    finally:
        led_cuarto.off()
        led_cocina.off()
        led_bano.off()

```